

FedWatcher

An Agentic System for FOMC Tone Extraction and Policy-Rate Nowcasting

Johnny Magon

Lorenzo Mottinelli

Leonardo Palanca

Sebastian Pasz

Programming in Finance II, Spring 2026 – Università della Svizzera italiana

June 12, 2026

GitHub: <https://github.com/LeoPalanca/FEDWatcher>

Live dashboard: <https://fedwatcher.ellep.it>

1 Project Plan

1.1 Problem

The U.S. Federal Reserve communicates monetary-policy intent through periodic FOMC statements and minutes. Markets re-price aggressively around these releases, but the textual signal embedded in the documents is high-dimensional, qualitative, and partially redundant with what is already implied by Fed-funds futures. *FedWatcher* asks a single, narrow question: *given the latest FOMC text and a current snapshot of macro and rates data, what is the most likely outcome of the next FOMC meeting (cut / hold / hike, in basis-point buckets)?*

1.2 Solution at a Glance

FedWatcher is an *agentic* pipeline made of three named agents that read from and write to a single SQLite database:

MonitorAgent

Scrapes new FOMC statements from the Federal Reserve website (and a synthetic FakeFed mirror used for scraper tests) and inserts them into the `documents` table. Minutes and speeches are filtered out upstream so the pipeline only operates on statements. Although this component does not use an LLM, we still refer to it as an agent to maintain consistency across the FedWatcher architecture.

AnalystAgent

Splits each document into rhetorical sections, asks an LLM to score the tone of each section on a structured rubric (overall tone, inflation assessment, labor-market assessment, forward guidance), and writes the result into `sentiment` and its variants (`sentiment2`, `sentiment3`, `sentiment_w`) used for ablations.

StrategistAgent

Applies a time smoothing on all the stored sentiments and joins the final sentiment row with macro features (CPI, core CPI, unemployment, 2y yield, policy rate) from FRED and produces a probability vector over the next-meeting outcome plus a short natural-language explanation.

A FastAPI service exposes the database read-only over HTTPS; a static HTML/JavaScript dashboard renders document feeds, macro charts and signal tables on top of that API.

1.3 Generic Project Criteria Coverage

The course allows any project that combines three of the listed elements with at least one *advanced* (starred) component. FedWatcher covers:

- **Cloud-based infrastructure:** deployed on a Debian VPS (Bavaria, OVH) behind Nginx; `systemd` supervises the API; `cron` drives all ingestion and analyst loops.
- **Large language model:** AnalystAgent uses an LLM (Anthropic Claude and OpenAI models via OpenRouter) for structured tone extraction; StrategistAgent uses an LLM for narrative synthesis.
- **Machine learning:** an ordered/multinomial nowcast over $\{-50, -25, 0, +25, +50\}$ basis-point buckets, trained on macro features and tone aggregates.
- **Non-trivial database:** SQLite with ~ 15 related tables (documents, multi-version sentiment, signals, weights, section weight calibration, market and macro data, policy moves).
- **Real-time data:** hourly cron polling of Fed pages and FRED series.
- **Advanced data visualization:** an interactive single-page dashboard with document feed, modal viewer, range/scale pickers, and per-table charts.
- **Own API:** a read-only FastAPI service with health, table, snapshot, and document endpoints.

1.4 Repository Layout

- `agents/` – runtime agents (analyst variants, dual-model analyst, `w_agent`, `strategist`).
- `scripts/` – ingestion, backfill, schema, and cron-wrapper shell scripts.
- `app/` – FastAPI app exposing the SQLite store.
- `fedwatcher/` – static dashboard (HTML, CSS, vanilla JS).
- `fakefed/` – synthetic Fed-like static site used as a scraper fixture.
- `db/schema.sql` – the canonical schema (single source of truth).
- `AGENTS.md` – contributor contract for human and AI collaborators.

2 Project Diary

The GitHub commit log is the canonical record of who did what and when; this diary is the editorial layer on top of it. Rather than narrating each commit, it groups the eight weeks of work into phases, each phase framed around the question the team was trying to answer and the artefact that survived from it. Individual authorship is acknowledged where a contribution shaped the project’s direction; routine commits (typos, cosmetic tweaks, in-flight reverts) are omitted. Author initials: **LP** = Leonardo Palanca, **SP** = Sebastian Pasz, **LM** = Lorenzo Mottinelli, **JM** = Johnny Magon.

2.1 Phase 1 – Bootstrap (April 18, 2026)

The first day was about answering one question: *can we get from “a Fed statement exists on a webpage” to “it lives in a row in our database”?* LP set the visual and documentation tone – a placeholder README, an architecture sketch, and an HTML mockup whose layout language survived every later iteration of the dashboard. SP built the Python side end-to-end in a single evening: requirements scaffolding, the first SQLite test harness, a scraper sandbox, the first **MonitorAgent** prototype, and the convention of keeping secrets out of the tracked tree. By the end of the day the loop worked end-to-end on a handful of statements, and the team adopted a commit-message style (descriptive subject, verified-by trailer) that held for the rest of the project.

2.2 Phase 2 – Architecture Consolidation (April 30, 2026)

After a pause to clarify the brief, the focus shifted from “does it run” to “does it have a contract”. LP rewrote `AGENTS.md` as a binding entry point for any future contributor – human or AI – and folded the architecture description into the README so the repository had a single source of truth for what each agent does. The most consequential move of the phase was operational rather than code: LP stood up **FakeFed**, a synthetic, statically served clone of the Federal Reserve website behind Nginx, so the scraper could be exercised end-to-end without touching the live Fed site. That decision paid off repeatedly later – every fixture-driven test, every demo, and every accountability backtest runs against it. SP cleared dead files from the bootstrap day so the next phase started from a clean tree.

2.3 Phase 3 – Analyst v1 and Macro Ingestion (May 5–13, 2026)

With ingestion stable, the next question was how to turn raw text into a number. JM shipped the first end-to-end **AnalystAgent**: segment each FOMC document into rhetorical sections (assessment, outlook, policy stance, voting record), ask an LLM to fill a structured rubric per section, write the result back into `sentiment`. LP introduced **OpenRouter** as a vendor-agnostic LLM gateway so the team could swap between Anthropic, OpenAI, and DeepSeek backends without code changes, and shipped the first FRED macro ingestion – a `macro_data` table plus a backfill command for CPI, core CPI, unemployment, and the 2-year yield. The lesson of this phase was that the monitor had to be source-agnostic: LP refactored it mid-May so the same code path ingested both the live Fed site and **FakeFed**, which removed the last temptation to write differential parsing logic.

2.4 Phase 4 – Multi-Analyst, Strategist and Automation (May 19–22, 2026)

The team intentionally produced several competing analyst implementations rather than committing to one early. SP shipped two “better analyst” iterations exploring alternative segmentation strategies and prompts; JM added a *dual-model* analyst that runs two LLMs in parallel and reconciles their outputs. Each variant writes to its own sentiment table (`sentiment`, `sentiment2`, `sentiment3`), and all of them survive in the production tree as testing baselines – the weight-aware `w_agent.py` that became the production analyst was selected on the strength of being able to tune section weights without re-running the LLM. JM fixed a weighted-score bug along the way where zero-weighted sections were polluting the average and biasing short documents toward neutral; that fix is the reason the current scoring formula explicitly excludes zero-scored sections from the average. LM shipped the first real **StrategistAgent**: EWMA-smoothed tone fed through an ordered-probit nowcast, with the divergence sign convention pinned (positive means the market prices a higher next rate than the tone model) and the FOMC historical-page backfill (back to 1994) moved into `scripts/` so a fresh install can rebuild the full corpus reproducibly. SP wired the whole pipeline onto cron with a bash wrapper layer, a 90-day rolling refresh window, and a sequence of glue scripts; JM added the final “run-everything” orchestrator that closed the loop.

2.5 Phase 5 – Productionization and Ops (May 22–27, 2026)

The fifth phase was about turning a working prototype into a service. LP replaced the static `snapshot.json` that originally powered the dashboard with a live read-only FastAPI service and gave the data explorer the labels and legends it needed to be self-explanatory. LM backfilled `FEDFUNDS` into `macro_data` to unblock the strategist’s tone-implied-rate calculation, and JM aligned `db/schema.sql` with the multi-weights analyst.

The defining incident of this phase was a schema-drift bug discovered the same week: two SQLite copies of the production database had silently drifted, with the API reading a complete `/opt/FEDWatcher/fedwatcher.db` while every cron writer was updating an incomplete `/home/programming/FEDWatcher/fedwatcher.db`. The fix – merging the two databases, re-pointing the API at the cron-written copy, and archiving the redundant tree – is unremarkable; the policy change is what matters. LP tightened `AGENTS.md` so that any contributor must treat

`db/schema.sql` as authoritative and must explicitly flag missing tables or columns rather than silently introducing them. That contract has been honoured in every subsequent schema change.

2.6 Phase 6 – Live Dashboard, Strategist Automation and the Accountability Experiment (May 27 – June 2, 2026)

LP swapped the v1 prototype dashboard for a live v2 dashboard wired directly to the FastAPI service. LM put the strategist on a 5-minute cron loop so each new statement triggers a fresh signal write, and JM landed a first attempt at a publicly visible track record: an **accountability** FastAPI router exposing hit rate, MAE, and Brier score over the strategist’s ordered-probit signals. Two methodological issues surfaced almost immediately and were fixed by JM in a follow-up commit. First, the original code diffed `policy_rate` around the meeting that had just happened, but the bucket probabilities target the *next* meeting, so each forecast $S(M)$ was re-anchored to $S(M+1)$ and the realised outcome was read from **FEDFUNDS** around that anchor month. Second, intermeeting and emergency statements (gap to the previous statement below 35 days) were filtered out of both the forecast and anchor positions, since they cannot fairly be graded against a model that targets scheduled meetings. LP added the academic-report draft and the dashboard footer linking to it; LM produced a math review of the report and unblocked the LaTeX build by adding `lmodern` and correcting a math-mode escape in the methods section.

2.7 Phase 7 – Design Pass, Deploy Refactor and Accountability Rollback (June 7, 2026)

The seventh phase was the design crystallisation pass. LM brought a full proposal design live in a single day – a §04 Method modal, a §05 Accountability section with KPI tiles and a recent-predictions table, additional hero-takeaway modals, a back-to-top control – and wired §05 to `/api/accountability` so the hardcoded placeholder numbers were replaced by live values. LP and JM repalettised the signal-divergence chart from green/red to two shades of purple, untangling the directional connotation of red/green from the tone-vs-market gap the chart actually shows. On the ops side LP introduced a deploy script using a restricted **programming** user with SSH connection multiplexing and realigned the backend deploy path with the **systemd** runtime tree, which made every later deployment a single command instead of a checklist.

The accountability section was then *removed* from production later the same day, in two commits by JM: the router was unmounted from `main.py`, and the §05 block, the navbar link, and the corresponding fetch logic were stripped from the dashboard. The reason is documented in the strategist’s own source: the ordered-probit β coefficients and cut points are explicitly flagged as “placeholders calibrated to economic priors... but MUST be refit on historical FOMC outcomes before being published” (`agents/strategist.py`, lines 25–29). Publishing hit rates and Brier scores against an unfitted model would have advertised numbers that reflect prior beliefs rather than measured performance. The code was preserved (`app/accountability.py`) so the same metrics can be brought back online unchanged once the strategist is refit on the historical corpus.

2.8 Phase 8 – Pipeline Hygiene, AI Copy and the Final Local Launcher (June 8–12, 2026)

The closing phase had no single objective; instead it tied off the loose ends so the project could be handed in as a coherent whole. **Pipeline scope** was narrowed across the entire codebase: LM removed minutes and speeches from the ingestion, scoring, and weights logic so the production path is unambiguously statement-only, and updated every reference – in the analyst variants, the monitor, `run.py`, and `update_weights.py` – to match. **The FRED monitor** was hardened by SP in three iterations: handling missing months by averaging adjacent observations, projecting one forward proxy month so the strategist always has a row for the latest document, and rationalising the **FEDFUNDS** column naming to use the existing `policy_rate` field.

The narrative pipeline – the AI-generated hero and §02 Breakdown copy – was completed by LM. The deterministic accountability and snapshot endpoints were already in place; what was

missing was a layer that turned those numbers into prose. LM shipped `/api/narrative` with a system prompt explicitly bounded to “write prose, do not invent numbers”, a process-wide cache keyed by latest signal id, and a deterministic fallback when no OpenRouter key is present. The dashboard was then taught to render the AI copy directly into the existing layout, with hawkish/dovish/neutral consistently italicised and rendered in the project palette. **FakeFed** graduated from a static fixture into a managed surface: LP added password-guarded `POST/DELETE /api/fakefed/statements` endpoints so synthetic statements can be published or removed from the dashboard side without manual file edits, then closed the loop by making file deletion cascade to the corresponding `documents`, `sentiment_w`, and `signals` rows. JM split the README into a conceptual document and a hands-on `USER_GUIDE.md` so new contributors have a single entry point for installation, the FastAPI surface, and the FakeFed workflow.

Several small lessons from earlier phases compounded into structural fixes. The recurring problem of popups colliding with the sticky topbar was solved by pinning every `.modal` and `.popover-overlay` below the topbar safe zone, rather than overriding individual modals. Interpolated rows in the §03 `macro_data` table were highlighted in the palette yellow with an explicit legend, so a reader can tell at a glance which months are FRED data and which are the strategist’s adjacent-month average. LM closed the phase by aligning `run.py` with the README architecture end-to-end: `python run.py pipeline -once` now runs the full chain (FRED → MonitorFed → FakeFed (optional) → Analyst → Strategist), with a `-rescore` flag for full signal regeneration and a DB migration script (`scripts/migrate_db.py`) that brings older databases up to the current schema without losing data. **The state at hand-in is the state a new contributor reaches by running a single command on a clean machine.**

2.9 What we kept and what we set aside

Across the eight weeks the team kept a small, stable toolchain: **Python 3** (FastAPI, `requests`, `beautifulsoup4`, `pandas`, `sqlite3`), **SQLite** for storage, **Nginx** and **systemd** on the VPS, and **vanilla HTML/CSS/JS** (Chart.js) for the dashboard, deliberately avoiding a frontend framework. Generative-AI tools and their usage are documented in Section 7.

The most important decisions the team did *not* make are also worth recording. The legacy analyst variants were never deleted – they remain in the codebase as testing baselines, since the choice of `w_agent.py` as the production analyst depended on their existence as comparison points. The accountability router was not deleted when the §05 section was removed from production; it remains ready to be re-enabled once the ordered-probit model is refit. ECB support remained a stretch goal and was never started: every hour spent on it would have been an hour not spent calibrating the model that actually powers the dashboard. These omissions are what made the eight-week schedule survive contact with reality.

3 Economics and Mathematics behind the FEDWatcher

3.1 Theory

The object of interest of this project is the outcome of the next FOMC meeting, defined as the change in the upper bound of the federal funds target range relative to the current one. In order to compute the prediction, we described the possible actions of the FED through five buckets:

$$Y_{t+1} \in \mathcal{Y} = \{-50, -25, 0, +25, +50\} \text{ basis points.}$$

Then, the estimation model is a Probit that follows this specification:

$$Y_t^* = X_t \beta + \epsilon_t,$$

and the conditioning set is

$$\mathcal{X}_t = \underbrace{(\tau_t^{(1)}, \dots, \tau_t^{(K)})}_{\text{LLM tone scores}} \cup \underbrace{(\pi_t, \pi_t^{\text{core}}, u_t, u_t - u_t^*, y_t^{2y}, r_t^{\text{ff}}, y_t^{2y} - r_t^{\text{ff}})}_{\text{macro and rates}}.$$

Here π_t and π_t^{core} are year-on-year headline and core CPI, u_t is unemployment, u_t^* is the natural rate (FRED NROU) so that $u_t - u_t^*$ is the unemployment gap, y_t^{2y} is the 2-year Treasury yield, r_t^{ff} is the current Fed-funds target, and $y_t^{2y} - r_t^{\text{ff}}$ is a crude proxy for the market’s pricing of forthcoming policy.

3.2 Tone Extraction

For each new document d , AnalystAgent splits the text into $K \approx 5$ sections using regex anchors trained on the structure of FOMC statements (assessment, outlook, policy stance, voting record). The LLM is then prompted with a strict JSON schema and returns, per section, an overall tone in $[-1, 1]$, an inflation assessment label (**cooling**, **stable**, **heating**), a labor-market assessment, a forward-guidance label (**dovish**, **neutral**, **hawkish**) and a confidence score. Section scores are aggregated to a single document tone by a calibrated weighted average:

$$\tau_t = \frac{\sum_k w_k \text{conf}_k s_k}{\sum_k w_k \text{conf}_k}, \quad w_k \text{ from } \text{section_weight_calibration},$$

where the section weights w_k are estimated by regressing historical document tone against the realized next-meeting outcome on the training window. The dual-model analyst runs two distinct LLMs in parallel and takes a confidence-weighted average to reduce single-model variance; the resulting tone is stored in `sentiment2/sentiment3`, and a weighted variant in `sentiment_w`.

3.3 Nowcast Model

The baseline nowcast is an *ordered probit* over j with cutpoints $c_1 < c_2 < \dots < c_5$:

$$Y_t^* = X_t \beta + \epsilon_t, \quad \Pr(Y_t = j \mid \mathcal{X}_t) = \frac{\exp \mathcal{X}_t \beta_j}{1 + \sum_{k=1}^J \exp \mathcal{X}_t \beta_k}, \quad c_0 = -\infty, \quad c_6 = +\infty,$$

The full specification is then defined as

$$Y_t^* = \beta_1 \cdot S_t + \beta_2 \cdot \Delta CPI_t + \beta_3 \cdot Ugap_t + \epsilon_t$$

where ΔCPI_t is the variation in the Consumer Price Index, $Ugap_t$ is the unemployment gap and ϵ_t is the error term. S_t represents the tone score, extracted by a text analysis performed by an LLM from all the statements and documents published by the FED. In particular, S_t is an Exponentially Weighted Moving Average (EWMA) defined as follows:

$$S_t = \alpha_t \cdot \text{tone_score}_t + (1 - \alpha_t) \cdot S_{t-1}, \quad \alpha_t = 1 - e^{-\lambda \Delta t}, \quad \text{where } \lambda = \frac{\ln(2)}{h}.$$

The first observation in our database seeds the series with $\alpha = 1.0$ and $S_{t-1} = S_t$. After some research and literature review, we decided to set h equal to 21. We believe this is the best option for our purposes the project objectives.

3.4 Computation of Tone Implied Rate and Signal Divergence

The final tone implied rate $\hat{r}_{t+1}^{\text{tone}}$ is computed by summing to the current Fed-funds target the sum of the probabilities of each bucket j multiplied by the corresponding basis points:

$$\hat{r}_{t+1}^{\text{tone}} = r_t^{\text{ff}} + \sum_j \Pr(Y_t = j \mid \mathcal{X}_t) \cdot \text{bp}_j.$$

The StrategistAgent then computes the divergence

$$\delta_t = r_{t+1}^{\text{mkt}} - \hat{r}_{t+1}^{\text{tone}}$$

where r_{t+1}^{mkt} is the market-implied next-meeting rate from Fed-funds futures or, when unavailable, from the 1m OIS and $\hat{r}_{t+1}^{\text{tone}}$ is the tone-implied next-meeting rate (a calibrated mapping from τ_t to basis points).

4 Data Science

4.1 Data Sources

All macro series come from FRED via the public REST API: DFEDTARU, DFEDTARL, DFF, FEDFUNDS, CPIAUCSL, CPILFESL, UNRATE, NROU, DGS2, SOFR. FOMC statements are scraped from federalreserve.gov and from the synthetic fakefed.ellep.it used for parser tests; minutes and speeches are excluded from ingestion. The historical statement backfill covers 1994–present.

4.2 Why SQLite

The schema (`db/schema.sql`) deliberately favors clarity over performance: every `sentiment*` table uses `document_id` as a foreign key to `documents`, and signals are reconstructable joins rather than materialized views. SQLite was chosen over Postgres because the entire dataset fits in ~4MB, the read pattern is dashboard-style (few writers, many readers), and a single file is trivially backupable and reproducible.

5 Sample Results and Reflection

5.1 Database Coverage

At submission the production database contains FOMC statements spanning 1994-02-04 to 2026-04-29, with approximately 8 statements per year (one per scheduled meeting) and sparser coverage before 2000 due to gaps in the public Fed archive. Minutes and speeches are deliberately excluded from ingestion — the pipeline only operates on statements. Macro coverage is monthly, ~317 observations per series, from 2000 onward. The dashboard at <https://fedwatcher.ellep.it> renders all of this live from the FastAPI service.

5.2 Tone Time Series – Qualitative Read

The aggregated tone series τ_t tracks the well-known qualitative arc of recent monetary policy: pronounced dovish dips around 2008Q4 and 2020Q1–Q2, a sustained hawkish plateau across 2022–2023, and a gradual softening starting in late 2024. Sharp single-meeting jumps in τ_t correspond, in every case we inspected, to known turning points (e.g. the December 2018 “patient” language, the March 2020 emergency cut, the November 2023 pause). This is a sanity check rather than a quantitative result: it shows that the LLM-extracted signal is not random and is broadly consistent with what an economist reading the same documents would write down.

5.3 Case Study and Tests

In order to validate all the pipeline, from the tone extraction to the prediction model results, and the functioning of the front/back-end, we uploaded a new FakeFed document with a strong hawkish tone. The LLM correctly identified the hawkish language and assigned a high positive value to the tone metric. The final results on the dashboard were consistent with our expectations, confirming that the system is correctly processing analyzing, and interpreting the input data. FakeFed was then kept online to give the possibility to test and verify the functioning of the FedWatcher without having to wait for a new Fed statement release.

6 Lessons Learned

A strong project structure and scope is the key. The complexity of the project grew much more than expected. During a control meeting we realized we were facing real limitations, especially with the AnalystAgent: a single script was not enough to handle the different prompt versions, the section-splitting logic, and the various output formats we wanted to test. We decided to split the AnalystAgent into three separate agents, each writing to its own sentiment table. This decision was crucial, but this solution and the ones implemented to face other challenges would not have been possible without the clear project structure and scope defined in the planning phase.

Schema first, code second. The biggest single source of bugs was schema drift between developer machines and the VPS. The team eventually adopted a hard rule, codified in `AGENTS.md:db/schema.sql` is authoritative; any table or column not declared there must not be read or written; if a feature needs a new column, the schema is updated in the same commit. Every subsequent integration error became easy to diagnose because the failing query named the missing column outright.

Test on multiple analyst agents. As we started the project we realized that we could not rely on the results provided by only one single analyst agent. For this reason we decided to build multiple analysts, testing different models (or combinations of models), prompts and section-splitting logic. This turned out to be a great decision, as it allowed us to compare the results of different approaches and to have a more robust and reliable pipeline.

A static dashboard scales further than you think. The dashboard at <https://fedwatcher.ellep.it> is hand-written HTML, CSS and one vanilla JavaScript file (`explorer.js`). It calls one or two FastAPI endpoints and renders everything client-side with Chart.js. No build step, no framework, no `node_modules`. Deploying a change is `scp` of a single file. For a project where the team rotates between four developers and one VPS, this was the right call.

Use AI agents for what they are good at. Generative AI was used most effectively for (a) writing the AnalystAgent prompt, (b) explaining unfamiliar pieces of the codebase to whichever team member touched them next, and (c) ops scripting (the May 27 database merge was driven by Claude Code through an SSH session). It was used least effectively for designing the schema – early LLM-suggested tables had inconsistent naming conventions and missing foreign keys, and the schema only stabilized after a human review pass.

Cron + SQLite + systemd is enough. Several alternatives were considered (Celery + Redis, Airflow, RDS) and all were rejected as over-engineering. Five cron lines and one systemd unit run the entire production pipeline. The full ops surface fits in less than 30 lines of configuration, which made the May 27 server consolidation a one-evening task rather than a sprint.

What we would do differently. (i) Write the schema and the AGENTS contract on day one, not day forty; (ii) ship the FastAPI service before the static-JSON snapshot, so the dashboard never has to migrate; (iii) integrate the divergence sign convention into a unit test, not a code review; (iv) commit more frequently with smaller diffs – some of our commits ended up bundling unrelated changes and are harder to attribute in this diary than they should be.

7 Generative-AI Acknowledgement

In accordance with the course brief, we acknowledge that generative AI was used throughout the project. The following tools were involved and the role of each is summarized for transparency.

Anthropic Claude (Claude Code, Opus 4.6 and 4.7)

Used as the primary pair-programming agent: code edits, refactors, AGENTS.md drafting, the May 27 server consolidation (database merge, systemd repointing, archive download), and the draft of this very document. Invoked through the Claude Code CLI, not the chat UI, so that the full conversation, tool calls and diffs are reviewable.

OpenAI GPT-4 / GPT-4o (via OpenRouter)

Used as the secondary LLM in the dual-model AnalystAgent and for spot checks of the Claude-produced JSON output. Access goes through OpenRouter so that the API key is provider-agnostic.

GitHub Copilot

Used opportunistically in-editor for line-level completions, mostly in `explorer.js` and the FRED ingestion scripts.

Foreign code and citations. The project depends on standard open-source libraries: `fastapi`, `uvicorn`, `requests`, `beautifulsoup4`, `pandas`, `numpy`, `scikit-learn`, and `Chart.js`. All are listed in `requirements.txt` or loaded from a CDN in `fedwatcher/index.html`. No code was copied verbatim from external tutorials or Q&A sites; where an approach was inspired by an online reference (e.g. the FOMC regex anchors), the comment block in the relevant file names the source. The FRED data is public-domain via the Federal Reserve Bank of St. Louis API; the Fed documents are public-domain.

Repository and reproducibility. The full source code, commit history, issues and pull requests are available at <https://github.com/LeoPalanca/FEDWatcher>. Installation and deployment steps are in `README.md`; the cron schedule and systemd unit are reproduced verbatim in `deploy/`. Anyone with the GitHub URL can recreate the entire pipeline on a fresh VPS in under an hour.